

# Pharmaceutical Economic Data Hub: Normalized Infrastructure for Drug Pricing, Regulatory, and Billing Analysis

Naimisha Daripineni, Jesus Jimenez, Po-Chun Kuo, Aadhesharjun Senthilkumar

## Abstract

The United States pharmaceutical data ecosystem is public but fragmented. Drug acquisition costs, product approval records, therapeutic equivalence designations, biologics references, National Drug Code (NDC) package identifiers, Healthcare Common Procedure Coding System (HCPCS) billing codes, and Medicare Part B payment limits are published by different agencies with different schemas, release schedules, and identifiers. In this project, we built the Pharmaceutical Economic Data Hub: a normalized PostgreSQL-backed system that ingests public pharmaceutical economic datasets, preserves source-level provenance, exposes authenticated FastAPI query endpoints, and provides a Next.js analyst interface for NDC lookup, CMS billing translation, price history exploration, data freshness checks, and price forecasting. The system integrates NADAC pricing, FDA Orange Book, FDA Purple Book, OpenFDA, CMS NDC-to-HCPCS crosswalks, and CMS Average Sales Price files through a modular ingestion pipeline and a central mapping strategy based on normalized NDC and application identifiers. The implemented system now moves beyond a notebook proof of concept into a working internal data product: the repository contains a versioned relational schema, idempotent ingestion modules, source quality metrics, role-based access control, administrative configuration, and two forecasting model families for NADAC price projections. The work also makes visible the practical limits of public-source integration, especially the dependence on identifier availability, schema drift, and incomplete overlap between small-molecule and biologic regulatory datasets.

## 1 Problem

Pharmaceutical pricing analysis requires connecting facts that are rarely stored together. A single real-world question, such as whether a drug has a lower-cost therapeutically equivalent alternative or how a purchased NDC should be billed to Medicare, often crosses several public data systems. NADAC provides pharmacy acquisition cost observations and weekly or monthly pricing updates. FDA’s Orange Book provides approved drug products, therapeutic equivalence codes, patents, and exclusivity information for many drugs. FDA’s Purple Book provides licensed biological product and biosimilar information. OpenFDA provides

structured NDC and application metadata. CMS publishes ASP pricing files and NDC-to-HCPCS crosswalks that map physical drug packages to billing codes and payment limits (CMS, 2026; FDA, 2026a, 2026c; Medicaid.gov, 2026; openFDA, 2026).

These resources are valuable individually, but they are not designed as a single interoperable analytical system. They differ in file formats, headers, code conventions, identifier granularity, update frequency, and historical semantics. NADAC records are organized around NDC-level drug package prices over time. Orange Book records are organized around applications and product-level therapeutic equivalence. Purple Book records are organized around biologics license applications. CMS crosswalks connect package NDCs to HCPCS billing codes, while CMS ASP pricing files attach payment limits to HCPCS codes. Even when two files refer to the same product, they may use a 10-digit formatted NDC, an 11-digit billing-normalized NDC, an NDA or ANDA application number, a BLA number, a product name, or a billing descriptor rather than a common key.

The absence of a shared operational layer creates several recurring problems. First, analysts must repeat one-off data cleaning and joining steps before answering each question. Second, ad hoc scripts often obscure provenance: after a price or mapping is transformed, it may be unclear which source file, release date, or row produced the result. Third, historical tracking is difficult because source agencies revise files at different cadences and often publish snapshots rather than a database with transaction-time semantics. Fourth, business workflows such as generic substitution analysis, billing translation, and price volatility monitoring require both normalized structure and time-indexed observations. Finally, relying on raw public files directly makes downstream tools fragile when a government source changes headers, file layout, date fields, or naming conventions.

The project objective was to build a durable source-of-truth database and API layer for pharmaceutical economic analysis. The hub needed to aggregate public datasets, normalize identifiers, preserve raw source rows, track ingestion runs, expose secure query endpoints, and support analytical use cases such as historical NADAC review and price forecasting. The final implementation delivers the infrastructure portion of that scope through a PostgreSQL-backed architecture, an OpenFDA-centered mapping strategy, historical NADAC analysis, authenticated API routes, a web UI, administrative controls, and forecasting workflows.

## 2 Prior Work

The project builds on three categories of prior work: public pharmaceutical data publication, relational data integration, and time-series forecasting.

Public agencies already publish many of the source datasets needed for pharmaceutical economic analysis. Medicaid’s NADAC file reports acquisition-cost benchmarks and is updated weekly, with monthly NADAC rate updates produced on a regular release schedule (Medicaid.gov, 2026). FDA’s Orange Book identifies approved drug products and includes therapeutic equivalence evaluations for multisource prescription drugs, making it an important reference for generic substitution logic (FDA, 2026a, 2026b). FDA’s Purple Book provides information about licensed biological products, biosimilars, interchangeable products,

and reference products (FDA, 2026c). OpenFDA exposes FDA datasets through structured APIs and annotates original records into machine-readable JSON, including the drug NDC endpoint used for product and package mappings (openFDA, 2026). CMS publishes quarterly ASP pricing files and NDC-to-HCPCS crosswalks for Medicare Part B drug payment analysis (CMS, 2026).

Prior practical approaches often treat these public datasets as separate sources. Analysts download the files they need, clean them in notebooks, and join on available identifiers for a specific task. This approach is fast for exploration, but it does not provide a durable source of truth. It also makes it hard to compare results across analysts because cleaning rules, NDC normalization, application-number normalization, and date handling may vary from one notebook to another. Commercial drug databases address integration more comprehensively, but they are proprietary, expensive, and generally opaque about reconciliation rules. For an academic or internal engineering project focused on transparency, an open implementation with explicit schema and transformation logic is preferable.

The database design follows the relational model introduced by Codd, in which data is represented through relations that can be queried independently of physical storage details (Codd, 1970). Normalization is especially relevant here because drug, package, applicant, pricing, equivalence, and billing data form many-to-many relationships. A denormalized single table would be convenient for a dashboard export but would duplicate identifiers, blur entity boundaries, and make historical changes difficult to audit. PostgreSQL also provides practical support for semi-structured source preservation through JSON and JSONB data types, allowing the system to keep original source rows while still indexing normalized query columns (PostgreSQL Global Development Group, 2026).

The forecasting component draws from established time-series and machine-learning approaches. ARIMA models are a standard baseline for univariate time-series forecasting (Hyndman and Athanasopoulos, 2021). Prophet was considered because it was designed for analyst-guided forecasting at scale and for settings where many related business time series must be modeled repeatedly (Taylor and Letham, 2018). LightGBM was also implemented as a global machine-learning model because gradient-boosted decision trees can use lagged prices, rolling statistics, and calendar features across many NDC histories (Ke et al., 2017). The relevant research challenge is not only whether a single model can forecast a stable drug price, but whether model classes behave differently under pharmaceutical conditions such as generic price volatility, policy shocks, patent or exclusivity changes, and sparse historical coverage.

### 3 Solution

The project solution is a modular data hub with four main layers: ingestion, storage, access, and presentation.

The ingestion layer is a Python package organized by source. It exposes a Typer command-line interface with source-specific commands for NADAC, Orange Book, Purple Book, OpenFDA, CMS crosswalk, and CMS ASP pricing ingestion. Common utilities nor-

malize NDCs, parse dates, compute SHA-256 file hashes, detect variable header rows, read CSV, XLSX, and pipe-delimited files, normalize application numbers, normalize BLA numbers, and preserve JSON-safe source rows. Each ingestion run creates an `ingestion_run` record with source name, as-of date, file hash, status, row count, source path, and notes. If the same source, as-of date, and file hash were already ingested successfully, the pipeline skips the run unless forced. This design makes repeated local and cloud runs safer and easier to audit.

The storage layer is PostgreSQL, managed through SQLAlchemy models and Alembic migrations (SQLAlchemy, 2026). The schema has a raw staging layer for each implemented public source and a normalized core for reusable entities. Raw tables include `raw_nadac`, `raw_orange_book_products`, `raw_purple_book`, `raw_openfda_ndc`, `raw_cms_crosswalk`, and `raw_cms_asp_pricing`. Every raw row stores the ingestion run ID, as-of date, created timestamp, selected normalized columns, and the complete source row in JSONB. Core tables include drug, applicant, therapeutic equivalence code, NDC product, NDC package, and NADAC pricing entities. Indexes focus on operational access paths: NDC11, product NDC, normalized application number, HCPCS, effective date, as-of date, and composite NDC/as-of or HCPCS/as-of lookups.

The access layer is a FastAPI service with Pydantic response schemas and SQLAlchemy service functions (FastAPI, 2026). The API includes authentication routes, admin routes, health checks, metadata routes, NDC lookup and search, NADAC price history, NADAC statistics, NDC price prediction, CMS crosswalk lookup by NDC or HCPCS, and CMS pricing lookup by NDC or HCPCS. API routes are protected by bearer tokens, and admin-only routes manage users and volatility threshold settings. Passwords are hashed with scrypt, JWT access tokens expire after a configured time window, and the web middleware redirects unauthenticated users to login.

The presentation layer is a Next.js application using the App Router and TypeScript (Vercel, 2026). It provides pages for dashboard overview, NDC analysis, CMS analysis, dataset exploration, metadata freshness, system health, admin tools, account settings, and login. Reusable components render data tables with filtering, sorting, pagination, CSV export, JSON expansion, date formatting, and column resizing. The NDC page supports direct NDC lookup, drug-name search, NADAC history visualization, volatility and stability summaries, and an embedded price prediction panel. The CMS page supports crosswalk and pricing workflows. The admin page supports user management and configuration of risk thresholds used in NADAC volatility labels.

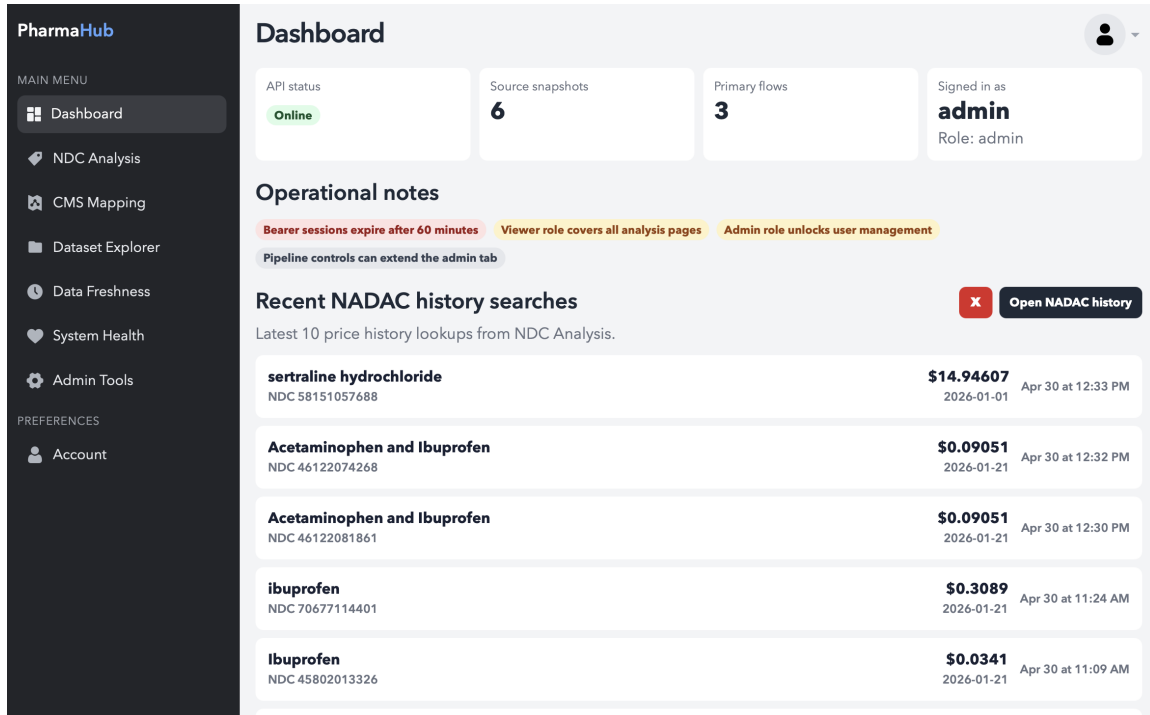


Figure 1: Dashboard page from the implemented web app, showing system status, source snapshot count, signed-in user context, and recent NADAC search activity.

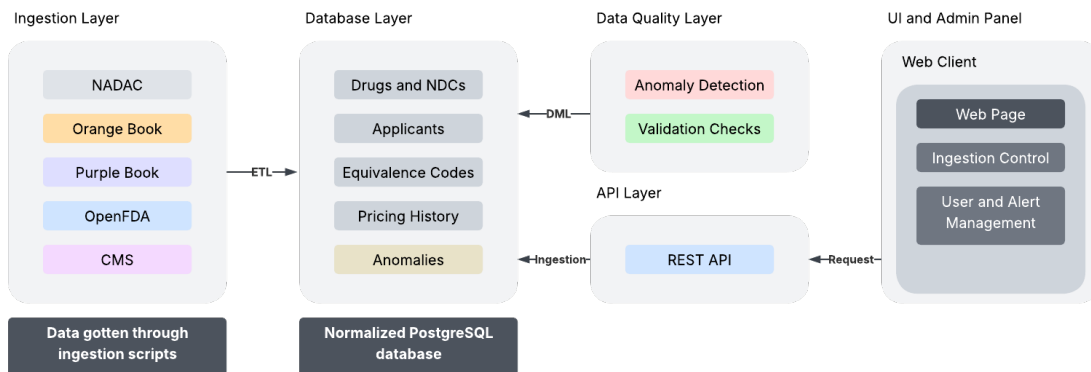


Figure 2: Repository architecture diagram showing public data sources, ingestion, PostgreSQL storage, FastAPI access, the Next.js interface, and planned Azure deployment components.

## 4 Method

### 4.1 Data Sources

The implemented system ingests public regulatory, pricing, and billing datasets. Table 1 summarizes the main files available in the project workspace and the analytical role of each source. Orange Book patent and exclusivity files were present in the workspace and inspected as regulatory context, but the current ingestion module and API use the Orange Book product file rather than those two supplemental files.

Table 1: Project data sources and observed local source-file scale. Counts reflect files inspected in the project workspace, not the total universe of each agency source.

| Source                      | Local scale                                                                                           | Role in the hub                                                                                                          |
|-----------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| NADAC                       | 1,730,427 rows; 33,774 unique raw NDCs; 57 as-of dates; effective dates from 2024-01-01 to 2026-02-02 | Historical pharmacy acquisition-cost observations used for price lookup, volatility analysis, and forecasting.           |
| FDA Orange Book products    | 47,588 rows; 26,844 application numbers                                                               | Approved drug products, therapeutic equivalence codes, ingredients, trade names, applicant names, and approval metadata. |
| FDA Orange Book patents     | 21,177 rows; 1,313 application numbers                                                                | Available local patent metadata for approved products; not loaded by the current API-facing ingestion module.            |
| FDA Orange Book exclusivity | 2,483 rows; 652 application numbers                                                                   | Available local exclusivity metadata for approved products; not loaded by the current API-facing ingestion module.       |
| FDA Purple Book             | 2,190 rows; 832 BLA numbers                                                                           | Biologic, biosimilar, interchangeable, and reference-product information.                                                |
| CMS ASP NDC-HCPCS crosswalk | 7,125 rows; 6,946 unique NDCs; 843 HCPCS codes                                                        | Mapping from NDC drug packages to Medicare Part B billing codes.                                                         |
| CMS AWP cross-walk          | 209 rows; 141 unique NDCs; 43 HCPCS codes                                                             | Supplemental crosswalk source for average wholesale price files.                                                         |

| Source              | Local scale                                | Role in the hub                                                      |
|---------------------|--------------------------------------------|----------------------------------------------------------------------|
| CMS OPPS cross-walk | 314 rows; 314 unique NDCs; 108 HCPCS codes | Supplemental outpatient prospective payment system crosswalk source. |
| CMS PrEP cross-walk | 37 rows; 37 unique NDCs; 5 HCPCS codes     | Supplemental crosswalk source for PrEP-related billing mappings.     |
| CMS ASP pricing     | 877 rows; 877 HCPCS codes                  | Medicare Part B payment allowance limits by HCPCS code.              |

## 4.2 Identifier Normalization

The central integration problem is identifier alignment. The system normalizes NDCs to an 11-digit representation because billing and package-level joins are most reliable at that granularity. The ingestion utility accepts 10-digit NDCs, hyphenated NDCs, and already-normalized 11-digit NDCs. For 10-digit values, it pads the labeler, product, and package segments to the standard 5-4-2 format. This is essential because public sources may provide values such as five-four-one, four-four-two, or five-three-two segment layouts.

Application identifiers are normalized separately. Orange Book application records distinguish NDA and ANDA records through application type and number. Purple Book records use BLA numbers. OpenFDA application identifiers may contain a prefix or only digits. The hub maps these into normalized keys such as NDA000000, ANDA000000, and BLA000000. This does not eliminate all ambiguity, but it prevents common failures caused by missing prefixes, inconsistent zero padding, or source-specific formatting.

OpenFDA is used as the central mapping layer when connecting NDC-level price records to application-level regulatory records. NADAC provides NDC-level prices but not complete application metadata. Orange Book and Purple Book provide application-centered regulatory information but not package-level acquisition costs. OpenFDA provides product and package NDC metadata with application numbers, brand names, and generic names. By first mapping NADAC NDCs to OpenFDA package NDCs, and then mapping OpenFDA application numbers to Orange Book or Purple Book records, the system avoids relying primarily on fuzzy name joins.

## 4.3 Ingestion and Provenance

Each source ingestion module follows the same pattern. It resolves the source file or directory, computes a file hash, checks whether an equivalent successful run already exists, creates an ingestion run, parses the source into a data frame, normalizes selected query columns, converts the full source row to JSON-safe values, inserts raw rows, and marks the run successful with a row count. If an exception occurs, the database transaction is rolled back and the run can be marked failed.

This design explicitly separates raw preservation from analytical transformation. Raw tables are not merely temporary staging tables; they are auditable records of what was present in a given source snapshot. The `source_row` JSONB column lets future analysts inspect fields that were not initially modeled. At the same time, targeted normalized columns support fast operational queries. This hybrid structure is a practical compromise between strict relational design and public files that change over time.

Header detection is important for CMS files because several spreadsheets include explanatory rows before the machine-readable header. The ingestion utilities scan early rows for required fields such as NDC, HCPCS, and payment fields before parsing. Purple Book loading also accounts for report-style preamble rows. These source-specific accommodations reduce fragility under mild schema drift.

## 4.4 Schema and API Design

The database schema is versioned through Alembic. The initial migration creates the ingestion run table, raw source tables, indexes, and core normalized entities. Later migrations add CMS units handling, authentication users, application settings, and configurable NADAC volatility risk cutoffs. The result is an explicit schema evolution history rather than a manually maintained local database.

The API layer separates routers from service functions. Routers handle HTTP path design, query parameters, response models, authentication dependencies, and error translation. Service functions perform SQL queries and response assembly. For example, the NDC service normalizes the input NDC, retrieves the most recent NADAC and OpenFDA records, gathers HCPCS mappings from CMS crosswalk rows, and returns a compact overview. The NADAC history endpoint groups raw records by as-of date, effective date, and price, while the statistics endpoint calculates monthly averages, month-over-month changes, volatility counts, and stability labels. The CMS service maps from NDC to crosswalk rows, derives matching HCPCS codes, and joins those codes to ASP payment limits.

## 4.5 Forecasting Method

The forecasting layer operates on monthly NADAC histories. Raw NADAC observations are grouped by effective month and averaged. Missing monthly periods are forward-filled after the first observed month so that models receive a regular monthly sequence. Two model families are exposed through the same API endpoint.

The ARIMA implementation uses an ARIMA(1,1,1) model as a classical univariate baseline. It requires at least three months of history and produces future monthly price forecasts. The LightGBM implementation loads a serialized model artifact and feature metadata. It requires at least 12 months of history and constructs lag features, three-month and six-month rolling means, a six-month rolling standard deviation, and month-of-year features. Forecasts are generated recursively: each predicted month becomes part of the known history used to predict the next month. Both models return target months, predicted prices, and summary statistics such as average predicted price, median, range, and total predicted percentage change.



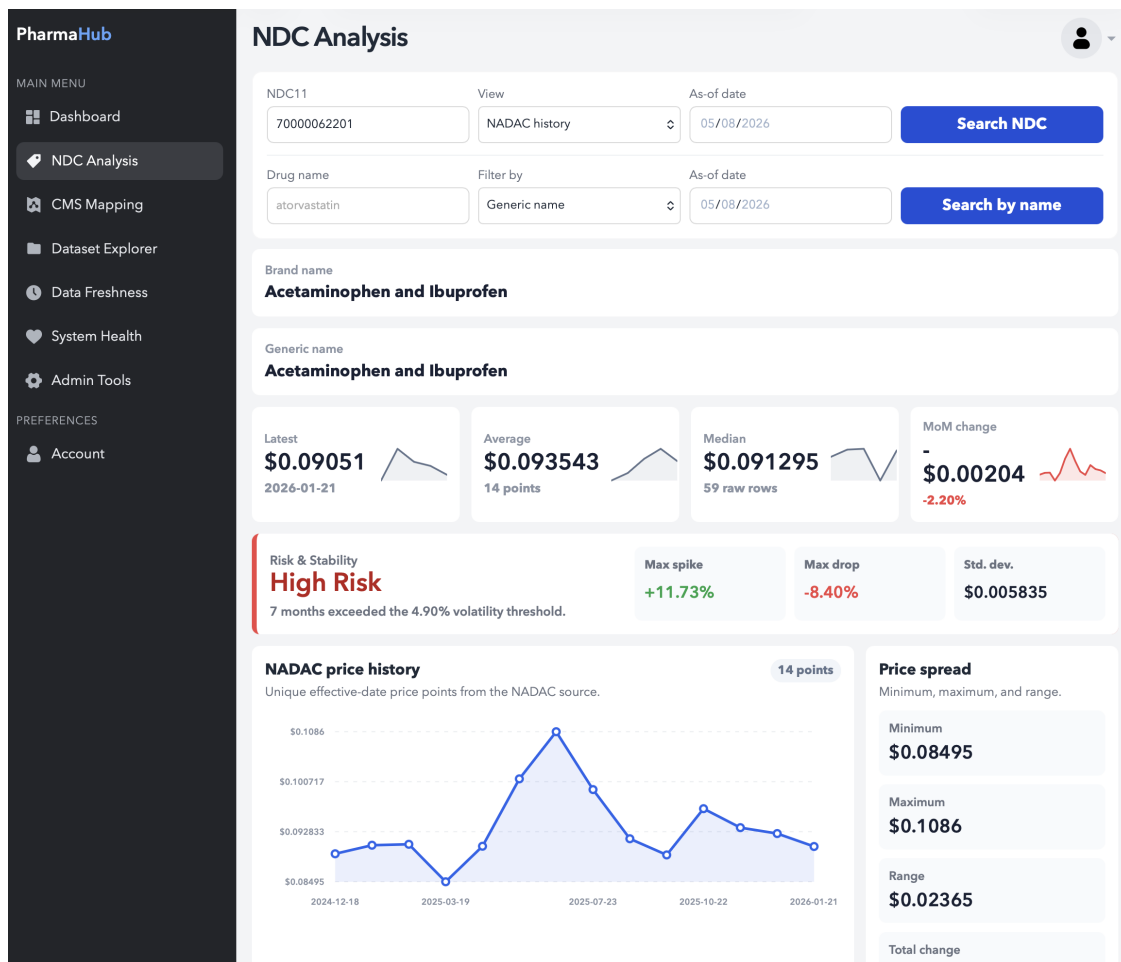


Figure 3: NDC analysis workflow from the implemented web app, including NDC or drug-name lookup, latest NADAC pricing, historical pricing, volatility summary, and prediction panel.

## 4.6 Use Cases

The project was evaluated through concrete workflows rather than only through schema inspection.

1. **NDC price history.** An analyst enters an NDC11 or searches by drug name, selects NADAC history, and receives historical price records, monthly statistics, stability labels, and a visual trend.
2. **Billing translation.** A user enters an NDC and receives CMS crosswalk rows, matched HCPCS codes, and Medicare Part B ASP payment limits where available.
3. **Source freshness.** A user opens the metadata page and checks available as-of dates by source to determine whether a dataset is current enough for analysis.
4. **Forecast exploration.** A user selects an NDC with sufficient history and compares 12-month LightGBM and ARIMA forecasts in the prediction panel.
5. **Administration.** An admin manages users and adjusts the NADAC volatility threshold and month-count cutoffs that control risk labels.

## 5 Results

The primary result is a working full-stack system that satisfies most of the major expected deliverables from the project description: automated ingestion modules, a normalized PostgreSQL schema, documented data quality metrics, and authenticated REST API endpoints. The implementation also adds a web UI, role-based user management, configurable volatility settings, and a forecasting API. A fully automated source-quality test suite remains future work.

### 5.1 Implemented System Outcomes

The repository is organized as a monorepo with separate application, pipeline, shared model, migration, infrastructure, data, and notebook directories. The backend is implemented in Python 3.12 with FastAPI, SQLAlchemy 2.x, Alembic, pandas, PyJWT, statsmodels, LightGBM, and supporting packages. The frontend is implemented in Next.js 14, React 18, and TypeScript. Docker Compose defines local PostgreSQL, API, web, and pipeline services. The README provides a fresh-machine command sequence for starting Postgres, running migrations, ingesting each source, starting the API, and verifying health and metadata endpoints.

The API currently exposes routes for:

- authentication: login, current user, and account update;
- administration: list, create, and delete users; get and update volatility settings;
- health and metadata: health checks and available source as-of dates;
- NDC analysis: lookup, name search, NADAC history, NADAC statistics, and price prediction;
- CMS analysis: crosswalk lookup by NDC or HCPCS and pricing lookup by NDC or HCPCS;
- samples: random or filtered data samples for quality assurance and exploration.

The UI turns these endpoints into analyst workflows. The dashboard summarizes system status and user context. The NDC page combines direct NDC lookup, drug-name search, price-history display, stability metrics, CSV export, and forecasting. The CMS page exposes multiple billing translation routes through a single form. The admin page manages user access and volatility defaults. These pages demonstrate that the API is not just technically callable but usable by a non-SQL analyst.

### 5.2 Data Quality and Integration

Repository quality notes document an important integration result: in a February 2026 quality run, 27,260 unique NADAC NDC11 values were available for matching; 85.10% matched to the OpenFDA crosswalk, 98.58% of matched values had a normalized application number, and 93.54% of matched application numbers joined to Orange Book records. Only 0.56% matched to Purple Book records, but this is expected because Purple Book covers biologics, while NADAC and OpenFDA are dominated by small-molecule drug products.

This low Purple Book rate is therefore an interpretive data-quality finding rather than an ingestion failure.

These metrics support the architectural choice to use OpenFDA as an intermediate mapping source. Direct matching from NADAC descriptions to Orange Book or Purple Book product names would likely be less reproducible because names vary by strength, route, package, manufacturer, abbreviation, and punctuation. The OpenFDA-centered approach does not solve missing identifiers, but it makes successful joins explainable: an NDC maps to an OpenFDA package NDC, then to an application number and related regulatory product records.

### 5.3 Analytical Results

The system supports four practical analytical outputs. First, NADAC statistics compute minimum, maximum, average, median, standard deviation, latest price, earliest and latest effective dates, price range, total percentage change, latest month-over-month change, volatile month count, maximum positive spike, maximum negative drop, and a stability label. The default volatility threshold is 5%, with moderate risk beginning after one volatile month and high risk after three volatile months. These settings are configurable by administrators.

Second, CMS billing translation is available through API and UI workflows. Given an NDC, the system can retrieve crosswalk rows, derive HCPCS codes, and retrieve ASP payment rows for those codes. This addresses a practical billing problem: drugs are often purchased and stored at the NDC package level but billed through HCPCS procedure or drug codes.

Third, a related goal was to support identification of lower-cost generic equivalents. The current system partially accomplishes this by ingesting Orange Book therapeutic equivalence codes and linking regulatory records to NDC-level price data through OpenFDA application identifiers. The retained notebook also includes exploratory generic-substitution logic that looks for AB-rated equivalents and, when needed, products with the same ingredient, route, and strength before joining to NADAC prices. This creates the data foundation needed for generic-equivalence analysis. However, that exploratory logic was not promoted into a complete API or UI recommendation engine that ranks equivalent products by price, dosage form, route, strength, and package constraints. That functionality remains future work.

Fourth, preliminary ARIMA experimentation was extended into an implemented prediction endpoint supporting both ARIMA and LightGBM. The experiments showed that ARIMA behavior varied meaningfully across drugs and was sensitive to parameter settings. The implemented system retains ARIMA as a baseline and adds LightGBM as a global model family that can use twelve monthly lag features, rolling-window features, and calendar-month features. Formal model benchmarking with held-out MAE, RMSE, calibration, and shock-period analysis remains incomplete, so the current result should be interpreted as a functional forecasting capability rather than a conclusive comparative study.

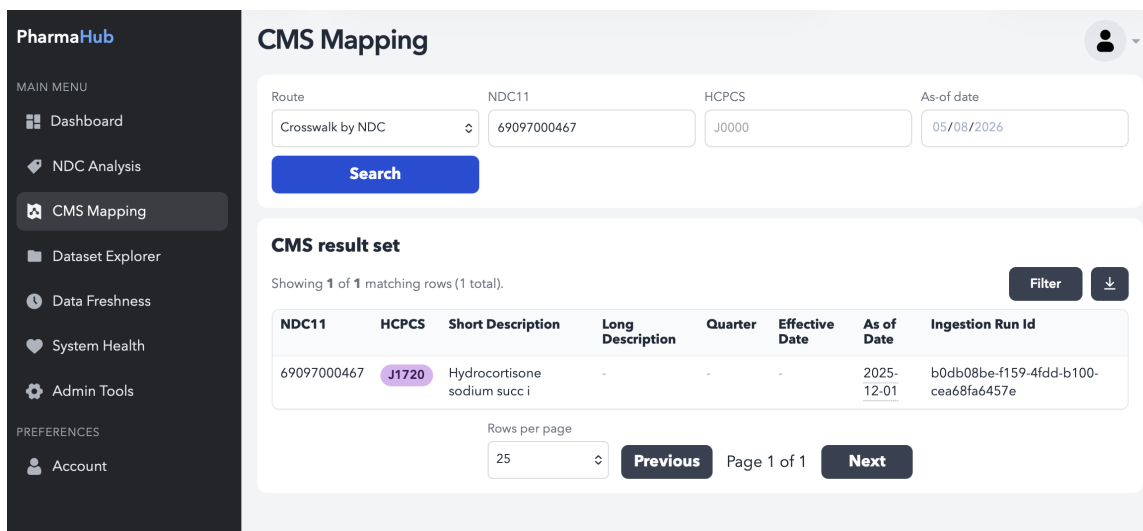


Figure 4: CMS billing translator workflow showing crosswalk and pricing lookup routes for NDC and HCPCS-based analysis.

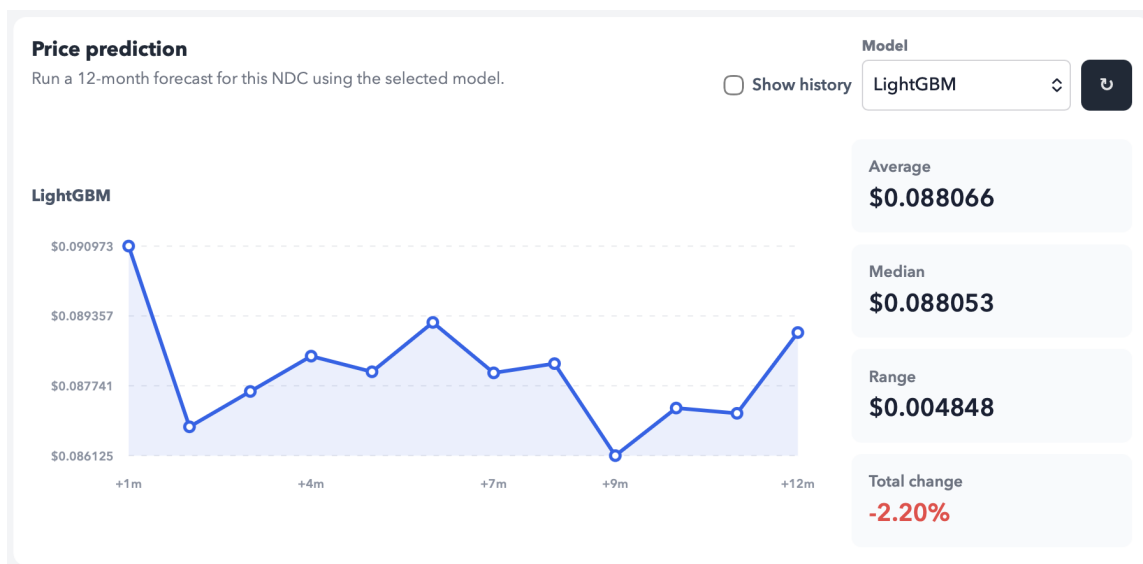


Figure 5: Forecasting panel from the NDC analysis workflow, showing model selection, forecast visualization, and summary statistics for NADAC price prediction.

## 6 Discussion

This project shows that a relatively small engineering team can convert fragmented public pharmaceutical data into a coherent internal data product, but it also shows why this problem is not solved by simply downloading more files.

One of the strongest design choices is the preservation of source-level provenance. Public healthcare datasets are revised, reformatted, and republished. If a downstream result changes, the system needs to make clear whether the change came from the source agency, a parser, a normalization rule, or a query. The `ingestion_run` table, file hashes, as-of dates, and JSONB source rows give the system an audit trail. This is especially important for price analytics because a forecast or volatility label may influence business decisions.

The second important design choice is using OpenFDA as a mapping layer. Public pharmaceutical identifiers exist at different levels of abstraction. NDCs identify listed products and packages, application numbers identify regulatory approvals, and HCPCS codes identify billing categories. A central OpenFDA mapping reduces ambiguity for many NDC-to-application joins. However, this dependency also creates a limitation: if OpenFDA lacks an NDC or application number, the hub may not be able to connect NADAC to Orange Book or Purple Book records without fallback methods. Future fallback logic may need controlled fuzzy matching, applicant-name normalization, and manual review queues.

The normalized schema is appropriate for integrity and extensibility, but analytical users often want denormalized views. The project addresses this through transform modules that build notebook-like data frames from raw database tables and through API response models that package related fields. A future production system should add materialized analytical views or incremental model tables so common dashboard queries do not repeatedly reconstruct the same joins.

Schema drift remains a central risk. The ingestion utilities already detect CMS header rows and handle several file layouts, but public sources can still change column names, add preamble rows, alter encodings, or publish revised files with the same apparent period. The project’s idempotency and provenance designs help detect these cases, but a production deployment should add automated schema validation, source-specific contract tests, and alerting when expected fields disappear or row counts move outside historical ranges.

The forecasting component is useful, but it is not yet a research conclusion. ARIMA provides a transparent baseline for individual NDC series, but pharmaceutical prices often include discontinuities, flat periods, abrupt generic competition effects, survey methodology effects, and payer-policy shocks. LightGBM can learn recurring patterns across many drugs, but it can also produce overconfident forecasts if trained on sparse or biased histories. A stronger forecasting study would need an explicit train-test design, drug stratification by stability class, evaluation across stable and shock periods, comparison against naive baselines, and interpretation of failure cases. The current system creates the infrastructure for that study by normalizing monthly price histories and exposing model outputs through an API.

Security is sufficient for a course internal tool but needs hardening before any real deployment. The implementation includes bearer-token authentication, bcrypt password hashing, role checks, and admin-only routes. However, the repository also seeds default development

users when configured, and the deployment notes are placeholders. A production version should require strong secret management, disable default credentials, configure HTTPS, implement audit logs for admin actions, and apply least-privilege database access.

## 7 Conclusion

The Pharmaceutical Economic Data Hub meets the project goal of building a centralized, queryable system for fragmented pharmaceutical economic data. It ingests and normalizes the major public sources used by the implemented workflows, preserves raw source rows and ingestion metadata, exposes authenticated API workflows, and provides an analyst-facing UI for NDC lookup, price history, CMS billing translation, metadata freshness, administrative control, and price prediction. The project moves beyond a notebook proof of concept by creating a versioned schema, modular ingestion architecture, API service layer, and reusable frontend.

The public project repository is available at <https://github.com/AadhesharjunS/Pharmaceutical-Economic-Data-Hub>.

The most important outcome is not any single endpoint or visualization, but the creation of a reproducible integration foundation. Once NADAC, OpenFDA, Orange Book, Purple Book, and CMS data can be queried through consistent identifiers and as-of dates, higher-level workflows become possible: generic substitution analysis, reimbursement translation, price volatility monitoring, forecasting, and policy research. The implemented system also clarifies the remaining challenges. Public data integration depends on incomplete identifiers, changing file formats, and uneven overlap across regulatory domains. Forecasting pharmaceutical prices requires more than fitting a model; it requires careful handling of sparse histories, policy changes, and structural breaks. The hub provides a practical foundation for those next steps.

## 8 Future Work

The first major extension is full Azure integration. The project already has a local Docker-based deployment path and Azure planning notes, but the next version should move the database, API, web application, secrets, logs, and scheduled ingestion jobs into managed Azure services. A cloud deployment would make the hub easier to share, monitor, and operate over time, especially if ingestion runs are scheduled rather than executed manually.

A second area for future work is a stronger data backfill workflow. Historical backfill could use automatic API calls, web scraping, or administrator-provided source files to collect older NADAC, CMS, FDA, and OpenFDA snapshots. Because those inputs may come from changing public pages or manually uploaded files, they should not be inserted directly into the production database. Instead, the system should stage them for administrator review, validate that the data is complete and viable, and run an anti-virus or security check before ingestion is approved.

A third extension is a more extensive drug classification and equivalence model. The current system provides the infrastructure for connecting NDCs, application numbers, therapeutic equivalence codes, and prices, but future work should build a richer model of drug relationships. That model should account for active ingredients, dosage form, route, strength, package details, therapeutic equivalence, biologic status, and pricing history so users can more confidently identify likely generic equivalents and lower-cost alternatives.

## Bibliography

- [1] Centers for Medicare and Medicaid Services. 2026. *ASP Pricing Files*. <https://www.cms.gov/medicare/payment/part-b-drugs/asp-pricing-files>
- [2] Codd, E. F. 1970. A relational model of data for large shared data banks. *Communications of the ACM* 13(6): 377-387. <https://doi.org/10.1145/362384.362685>
- [3] FastAPI. 2026. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- [4] Hyndman, R. J., and Athanasopoulos, G. 2021. *Forecasting: Principles and Practice*. 3rd ed. OTexts. <https://otexts.com/fpp3/>
- [5] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. 2017. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems* 30. <https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>
- [6] Medicaid.gov. 2026. *National Average Drug Acquisition Cost*. <https://www.medicaid.gov/medicaid/nadac>
- [7] openFDA. 2026. *Drug NDC API*. <https://open.fda.gov/apis/drug/ndc/>
- [8] PostgreSQL Global Development Group. 2026. *PostgreSQL 16 Documentation: JSON Types*. <https://www.postgresql.org/docs/16/datatype-json.html>
- [9] SQLAlchemy. 2026. *SQLAlchemy 2.0 ORM Documentation*. <https://docs.sqlalchemy.org/20/orm/>
- [10] Taylor, S. J., and Letham, B. 2018. Forecasting at scale. *The American Statistician* 72(1): 37-45. <https://doi.org/10.1080/00031305.2017.1380080>
- [11] U.S. Food and Drug Administration. 2026a. *Approved Drug Products with Therapeutic Equivalence Evaluations: Orange Book*. <https://www.fda.gov/drugs/drug-approvals-and-databases/approved-drug-products-therapeutic-equivalence-evaluations-orange-book>
- [12] U.S. Food and Drug Administration. 2026b. *Orange Book Preface*. <https://www.fda.gov/drugs/development-approval-process-drugs/orange-book-preface>
- [13] U.S. Food and Drug Administration. 2026c. *Purple Book: Lists of Licensed Biological Products with Reference Product Exclusivity and Biosimilarity or Interchangeability Evaluations*. [FDA Purple Book page](#).
- [14] Vercel. 2026. *Next.js Docs: App Router*. <https://nextjs.org/docs/app>